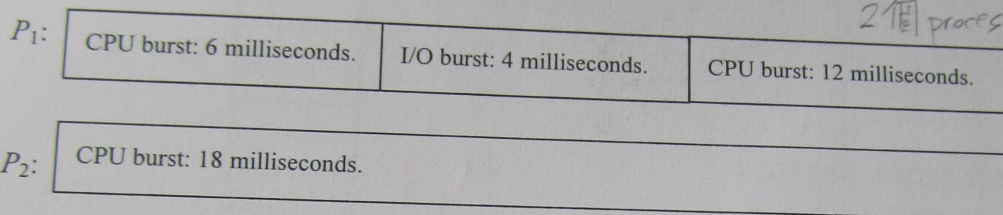
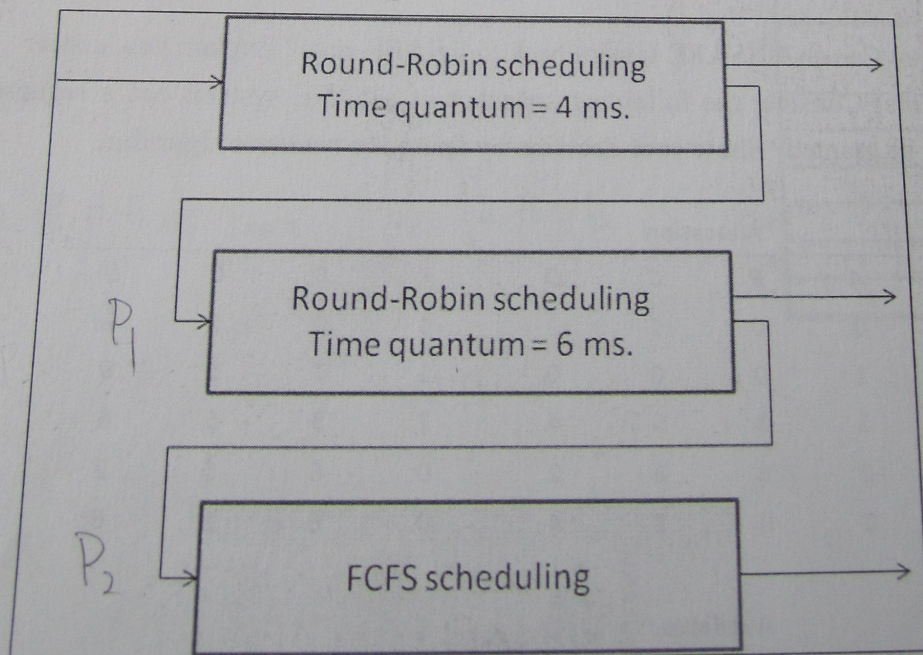


Operating Systems – Final Examination
June 21, 2011
Total: 100 points

1. Explain the following terms:
 - (a) (2 points) Turnaround time
 - (b) (2 points) Race condition
 - (c) (2 points) Deadlock
 - (d) (2 points) External fragmentation
 - (e) (2 points) Demand paging
2. Scheduling – Consider the following two processes P_1 and P_2 . P_1 consists of two CPU bursts and one I/O burst. P_2 has one CPU burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively.



- (a) (10 points) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that a high-level process will preempt a low-level process and a process will be scheduled to a lower level if it is preempted.



- (b) (10 points) What is the average waiting time for the scheduling algorithm?

3. Synchronization - With the support of synchronization hardware, solutions of the critical-section problem can be designed in a simple manner.

(a) (15 points) Complete the following n -process critical-section solution using the TestAndSet atomic instruction.

```
boolean TestAndSet (boolean *target) {
    boolean rv = *target;
    *target = true;
    Return rv;
}
```

// shared variables 'lock' and 'waiting[n]' are initialized to
// false

do {

CRITICAL SECTION

REMAINDER SECTION

} while (TRUE);

waiting[i] = true;
while (waiting[i] == true && lock == true)
 TestAndSet(&lock);
waiting[i] = false;
int j = (i+1) % n;
while (waiting[j] == false && j != i)
 j = (j+1) % n;
if (i == j)
 lock = false;
else waiting[j] = false;

(b) (10 points) Show that the code fragments satisfy the three requirements of critical section solution.

4. Deadlocks:

(a) (5 points) Can an **UNSAFE** system back to a **SAFE** state? Explain you answer.

(b) (10 points) Consider the following snapshot of a **SAFE** system; can a request (0,4,2,0) from p_1 be granted? Show your decision by using the banker's algorithm.

	Allocation 使用中				Max 最多需要			
	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2
P_1	1	0	0	0	1	7	5	0
P_2	1	3	5	4	2	3	5	6
P_3	0	6	3	2	0	6	5	2
P_4	0	0	1	4	0	6	5	6

Available 可用			
A	B	C	D
1	5	2	0

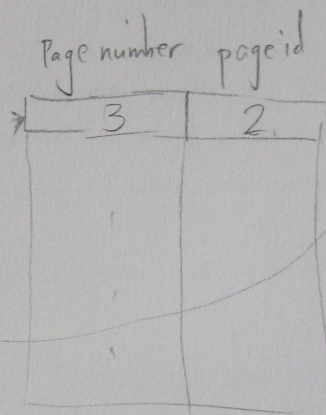
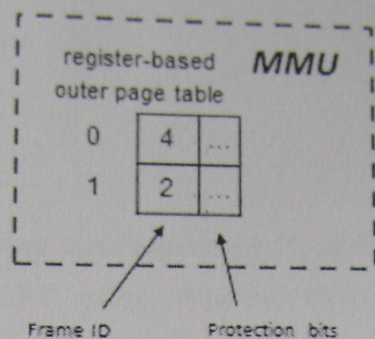
5. Paging – Consider a paging system that supports 4-bit logical address and has 32 bytes physical memory. The size of a page is 4 bytes.

- (a) (5 points) If the size of a page table entry is 2 bytes (first byte for frame number & second byte for protection bits), then how many bytes are required for a page table?
- (b) (10 points) The following figure shows the physical memory content and the outer page table (two-level paging) of a process. Fill the logical content of the process.

logical
address content

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

4bit



0	4	
1	2	
2		
3		
4		
5		
6		
7		

Frame ID

physical
address content

0	0
1	3
2	2
3	5
4	7
5	0
6	2
7	5
8	0
9	3
10	7
11	5
12	4
13	1
14	7
15	3
16	3
17	4
18	5
19	6
20	8
21	10
22	16
23	15
24	2
25	4
26	6
27	8
28	11
29	7
30	9
31	13